
01

Business Understanding



The Business Problem

The client, **Banca Massiccia**, is a large Italian bank seeking to reduce losses from bad loans. Their goal is to get better predictions of the **one-year probability of default (PD)** for corporate borrowers.

This project solves three distinct problems for the bank:

- **Financial Problem:** The bank aims to optimise their **risk-based pricing**. Without a reliable and calibrated PD, they cannot accurately price credit risk, which leads to financial losses from under-pricing risky borrowers and opportunity costs from over-pricing safe borrowers who then move to competitors.
- **Operational Problem:** Although statistical methods are used, the current underwriting process still depends heavily on human judgment and manual review, making it slow, subjective, and difficult to reproduce consistently. The bank needs an automated, objective, and consistent PD-generation tool to support faster and more reliable decision-making.
- **Regulatory Problem:** International accounting standards such as **IFRS 9** require banks to calculate Expected Credit Loss (ECL), a formula that mandates a statistically valid and properly calibrated PD. Compliance with these standards is non-negotiable, making a robust PD model essential for accurate provisioning and regulatory reporting.

Client: Banca Massiccia

Goal: Predict 1 Year Probability of Default (PD)

Financial Problem: Need Reliable PD for Risk-Based Pricing

Operational Problem: Heavy Human Judgment, Slow and Inconsistent

Regulatory Problem: IFRS 9 Requires Calibrated PD for ECL

How Our Solution Precisely Addresses the Problem

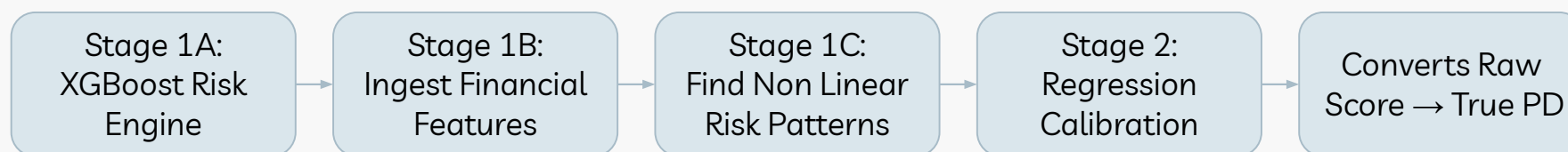
A simple "approve/deny" (classification) model would **fail** to solve the bank's problem. They don't just need a *ranking*; they need a *real, usable probability*.

Our solution is a **two-stage, finance-aware pipeline** that delivers the exact, usable PD the bank requires.

1. **Stage 1: XGBoost Risk Engine:** We use an XGBClassifier to ingest our **12 final financial features** (like altman_z and cfroa_w). This model excels at finding the complex, **non-linear risk patterns that traditional models miss**, achieving a high discriminatory power (AUC: **0.837**).
2. **Stage 2: Isotonic Calibration:** The raw XGBoost score is not a real probability. We feed that score into our custom PDModel Calibrator (which uses Isotonic Regression) to convert it into a **true, reliable PD**.

This two-stage model directly solves all three business problems:

- The **calibrated PD** is the direct input for their ECL and risk-based pricing formulas.
- The entire pipeline is automated in harness.py, solving the slow, manual operational problem.



Our Hybrid Approach vs. Past Work

Our approach is a hybrid that is superior to both the traditional finance and the "ML-only" methods, which both fall into critical traps.

- **Traditional Finance Approach (e.g: Logistic Regression):** This is the long-standing industry standard.
 1. **Good Idea We Kept:** Traditional models are grounded in financial theory. We preserved this strength by engineering finance-aware variables, such as the Altman Z-score, as core inputs.
 2. **The Trap: Logistic regression is linear.** It cannot capture the nonlinear interactions that drive real credit risk (e.g., high leverage is acceptable if cash flow is strong, but dangerous when cash flow is weak).
- **"ML-Only" Approach (e.g: Raw XGBoost):** Many modern FinTechs fall into this trap.
 1. **What They Do:** They use a powerful algorithm like XGBoost to achieve high discriminatory power (e.g., AUC 0.837) and stop there.
 2. **The Trap:** The model remains a black box, and its raw scores are uncalibrated. Without calibration, the output cannot be used for pricing or IFRS-9 ECL calculations. They build a strong engine but do not connect it to the controls the bank actually needs.
- **Our Hybrid Approach:** Our Solution combines the strengths of both worlds.
 1. **Accuracy:** We use XGBoost to capture the complex, non-linear relationships that traditional models miss.
 2. **Interpretability:** We restrict the model to only 12 finance, aware features, preserving transparency and avoiding "feature bloat."
 3. **Business Usability:** We add the PDModel Calibrator, converting raw XGBoost scores into a **true, reliable, calibrated PD** suitable for pricing, ECL, and regulatory reporting.

This "last-mile" step is what makes our PDs not only accurate but also **actionable** for the bank.

02

Problem Formulation

Section Overview

- Define what “default” means in our dataset and how we constructed the 12-month prediction target.
- Specify the prediction horizon the bank cares about and align the setup with how financial data becomes available in practice.
- Structure the data so the model learns from information that would realistically be known at underwriting time.
- Incorporate financial intuition to ensure the model captures economic drivers of distress rather than patterns in raw accounting values.

Defining the Prediction Problem

Goal : Estimate the probability that a borrower will default within the next 12 months using only the financial information that would have been available at the time of the lending decision.

How we defined the prediction task

- Firms in Italy typically publish approved financial statements 120–180 days after fiscal year-end
- To reflect this reporting lag, we defined an **“available date”** = *statement date + 4 months*.
- A firm-year is labeled as a **default** if a default event occurs **within 12 months after this available date**.
- This means the model predicts from “available date → 12 months ahead,” not from December to December, ensuring the prediction window reflects real underwriting timing.
- This comes under a **Supervised Learning Task** where the model will learn to predict default based on the given financial variables of a given year and the target variable that i.e whether a firm will default or not within a year.

Why this setup matters

- Aligns the model with the bank’s operational question:
“Given the information available today, what is the default risk over the next year?”
- Avoids look-ahead bias and prevents the model from benefiting from future information.
- Produces predictions suitable for real-world lending, pricing, and risk-management use.

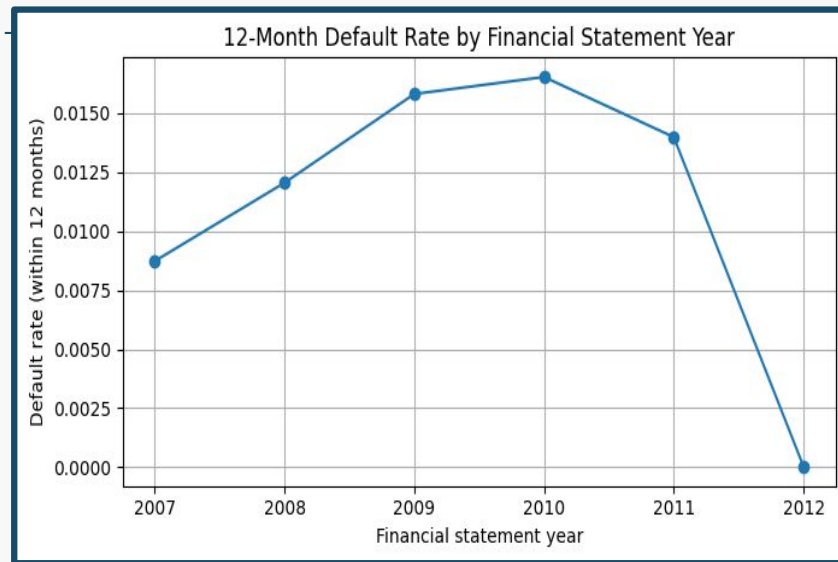


Figure 1

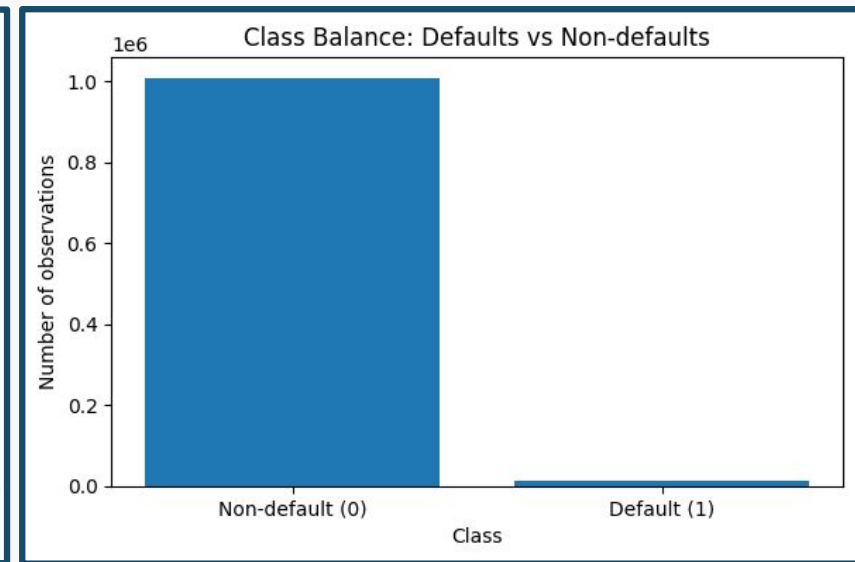


Figure 2

Figure 1: 12-month default rate by financial statement year. The default rate rises through 2009–2010 and declines afterward. The overall default rate in the dataset is 1.09%.

Figure 2: Class distribution of the target variable. The dataset is highly imbalanced, with 1,009,771 non-default observations and 11,156 defaults.

Importance of Financial Theory

Why not use raw financial statement values?

1. Raw items (assets, liabilities, revenues) scale with firm size and are not comparable across firms.
2. Default risk is tied to economic conditions, solvency, liquidity, profitability, not the absolute magnitude of balance-sheet items.
3. Financial ratios normalize values and reveal structural relationships that theory links to distress.

What the academic literature shows

1. Altman (1968): Liquidity, leverage, and profitability ratios discriminate strongly between bankrupt vs. non-bankrupt firms.
2. Ohlson (1980): Leverage, liquidity, and performance ratios are statistically significant predictors of corporate failure.

Implication for our model

1. **Ratio-based features expose the economic signals** associated with credit deterioration (indebtedness, cash sufficiency, earnings power).
2. **Ratios reduce noise and size effects**, enabling the model to learn meaningful, generalizable patterns rather than firm-size artifacts.

Incorporating Financial Intuition

We structured our feature engineering around five financial dimensions that prior empirical studies have consistently linked to corporate default risk.

1. Leverage and Solvency :

- When **leverage is high** (more debt relative to equity), a small decline in earnings can wipe out equity → **PD increases**.
- Strong equity and **lower leverage** give a bigger loss-absorbing buffer → **PD decreases**.

2. Profitability & Returns

- **Low or negative profitability** means the firm cannot generate enough internal cash to service debt and invest → **PD increases**.
- Higher, stable returns (ROA, ROIC, margins) indicate stronger earnings power and resilience to shocks → **PD decreases**.

3. Liquidity & Debt-Service Capacity

- **Weak liquidity** (low cash, low cash-to-assets, low interest-coverage) means even a temporary shock can cause missed payments → **PD increases**.
- **Ample liquid assets and comfortable interest coverage** give the firm room to absorb timing mismatches in cash flows → **PD decreases**.

Incorporating Financial Intuition

4. Cash-Flow Stability / Volatility

- **Highly volatile** or **consistently low operating cash flow** makes it harder to meet fixed obligations in bad periods → **PD increases**.
- **Stable, predictable operating cash flows** support regular debt service and reduce reliance on external financing → **PD decreases**.

5. Asset Structure & Debt Structure

- A **low share of tangible assets** limits collateral and recovery value; **heavy reliance on short-term debt** raises refinancing risk → **PD increases**.
- **More tangible, pledgeable assets** and a **higher share of long-term debt** reduce rollover risk and improve recovery prospects → **PD decreases**.

These dimensions guided which ratio families we created and ultimately supported our selection of the 12 final features used in the model.



Why Our Approach Outperforms Raw Data Mining

1. Translates accounting data into risk-relevant signals

- Ratios explicitly capture solvency, liquidity, profitability, and cash-flow strength, core economic drivers of default.
- Raw values do not reflect these mechanisms.

2. Encodes causal structures found in credit-risk research

- Capital structure, earnings volatility, and cash-flow coverage have been repeatedly validated as predictors of distress.
- Ratio-based features allow the model to learn from these relationships.

3. Reduces overfitting and improves generalization

- Raw accounting values vary widely across industries and firm sizes.
- Ratios stabilize scale differences and produce features that behave consistently across the dataset.

4. Supports interpretability and model credibility

- Ratios provide intuitive explanations (“PD increases when leverage rises”).
- Interpretability is essential for governance, validation, and risk-management use cases.

5. Aligns the prediction task with real underwriting constraints

- By using financially normalized, theory-driven features, the model is more aligned with how risk is assessed and therefore more deployable.

Key references

- Altman (1968), Ohlson (1980), Beaver (1966),



Section Overview

In this section we,

- **Describe the dataset provided by Banca Massiccia**, what each firm-year row contains, the types of financial statement variables included, and the overall structure and coverage of the data.
- **Identify data quality issues relevant for modeling**, including missing values, variables with limited availability, and patterns in missingness between defaulting and non-defaulting firms.
- **Explain which variables were viable for constructing financial ratios and which were excluded**, based on completeness, stability, and economic relevance, setting the foundation for later feature engineering.

03

Data Understanding



Dataset Summary

Dataset provided by Banca Massiccia:

- Annual financial statement data for Italian firms, provided at the firm-year level.
- Each row represents **one firm in one fiscal year** (“firm-year”) and this is our unit of analysis.
- Contains **44 financial statement variables** (assets, debt, revenues, profitability measures, cash flow items, etc.) plus identifiers (id, sector, legal structure, city).
- All monetary quantities are reported in **thousands of euros**, as specified in the original documentation.
- Coverage includes only:
 - Firms with **> €1.5M in assets**
 - **Non-financial** industries
- The training dataset includes historical default dates up to **12/31/2012**.
- The holdout dataset contains the same structure but **no default labels**.

What this means for modeling

- The dataset provides the raw accounting variables required to construct financial ratios (e.g., leverage, profitability, liquidity), but not all line items are equally complete or consistently available.
- Each firm appears across multiple years, with varying lengths of history.

The Target Variable Challenge

The single most problematic aspect of the raw data is the **lack of a target variable**.

- We were not given a simple "Default = 1 / No-Default = 0" column.
- We were only given def_date, which is just a date.
- **Problem:** To build a 12-month prediction model, we *must* create a target variable by looking *forward* 12 months from a given fs_year.
- **Business Insight (The Lag):** We must also account for the **Information lag**. A financial statement from fs_year2008 is not published and available to the bank until several months later.
- **Conclusion:** This *understanding* that our target is not given and that we must account for a time lag is the single most important finding. It directly motivates the way we setup the problem as explained in the problem formulation.

Potential Biases & Limitations

The data is *not* a random sample of all businesses. It has severe, known biases that strictly limit our model's scope:

- **1. Size Bias:** The data **excludes all firms with < €1.5MM in assets**. This is a major sampling bias. Our model will have no knowledge of small businesses or startups.
- **2. Industry Bias:** The data **excludes all finance and insurance firms**. Their balance sheets are fundamentally different, so our model is *only* valid for non-financial corporations.
- **3. Geographic Bias:** The data is from **Italian firms only**. The financial patterns it learns are specific to the Italian economy and cannot be applied to other countries.
- **4. Temporal Limitation:** The training data for defaults ends in 2012. This means the model has not been trained on recent economic events (e.g., a post-2012 high-interest-rate environment), which creates a risk of **model drift**.

Inherent Data Quality Problems

Finally, we understood from the raw train.csv that the data was "dirty" and presented three classic data mining challenges:

- **Severe Class Imbalance (Problematic):** Defaults are rare events. The raw data has a very low percentage of default (1s) vs. non-default (0s). This is problematic because a naive model will just predict "0" for everyone and achieve high accuracy. This *understanding* motivates our choice to use `scale_pos_weight` in the modeling step.
- **Extreme Outliers (Problematic):** Financial ratios (like ROE) are famously "noisy." A firm with tiny equity can have an ROE of +50,000%. These extreme values are not real economic signals; they are mathematical noise that can break a model. This *understanding* motivates our "Winsorization" step in Data Preparation.
- **Pervasive Missing Data (Problematic):** Many firms had NA for key financial items. Simply dropping these rows would have destroyed thousands of samples. This *understanding* motivates our imputation and feature engineering in Data Preparation.

Evidence from the data:

Return on Equity (roe): Missing for **only 6.5% of non-defaulting firms**, but missing for a staggering 44.3% of defaulting firms.

Financial Margin (margin_fin): Missing for **only 3.5% of non-defaulting firms**, but missing for 34.0% of defaulting firms.

Debt Variables (e.g., debt_fin_st): Showed moderate missingness (around 14%) for defaulted firms, further indicating distress.

Conclusion: Because of this imbalance, we decided that dropping rows wasn't a viable option, and would have to construct ratios and missing values to ensure a PD prediction for all types of data.



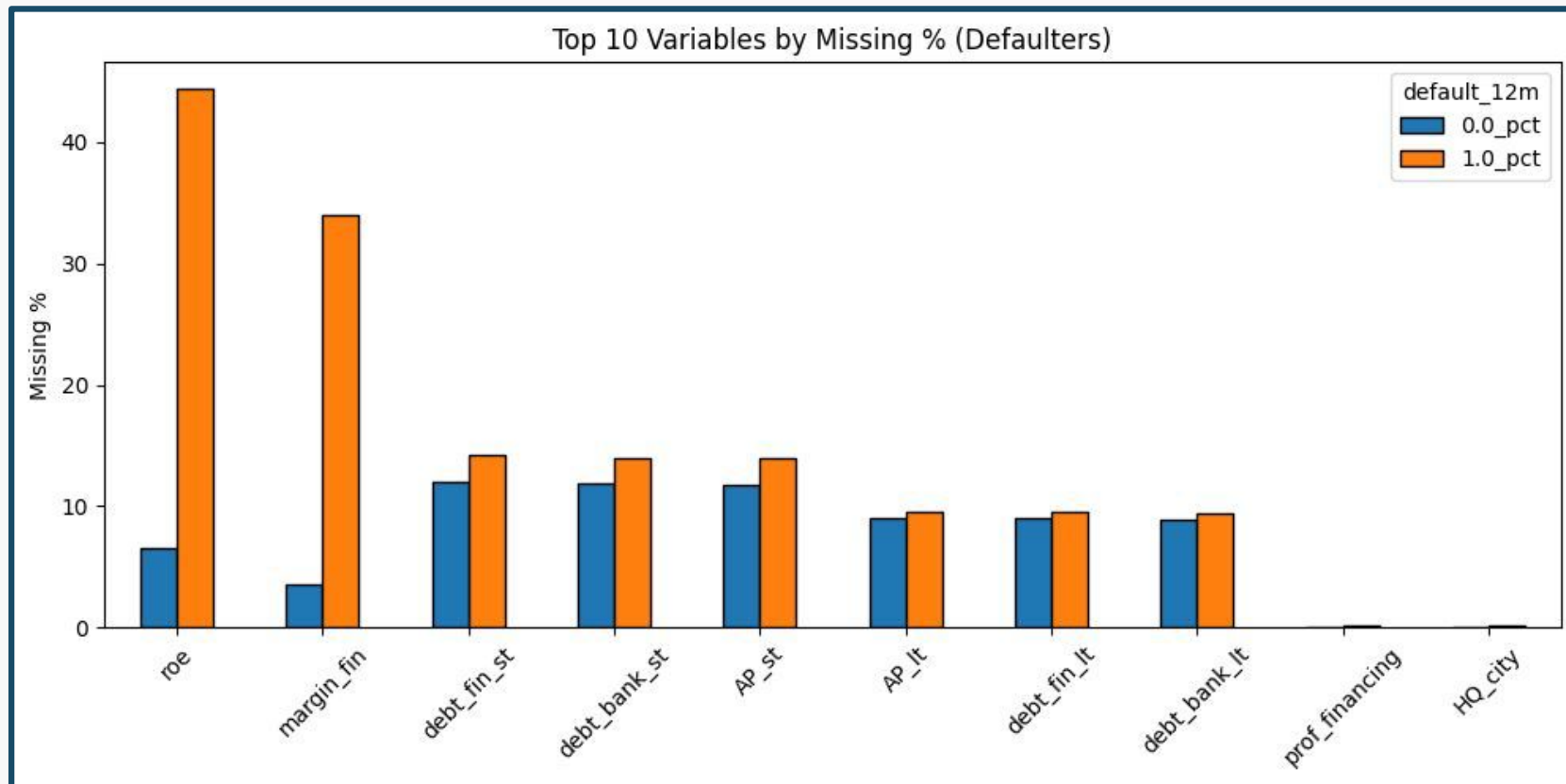


Figure 3: Plots the % of Missing Values Between Defaulters and Non Defaulters

04 Data Preparation

Section Overview

In this section we cover

- How core accounting quantities were reconstructed
→ Rebuilding assets, liabilities, and equity from components when reported values were missing.
- How missing and inconsistent values were handled
→ Identifying gaps, applying targeted imputations, and ensuring consistency across firm-years
- How financial ratios were constructed from cleaned inputs → Using reconstructed balance-sheet and income-statement items as stable denominators and numerators
- How final features were normalized and bounded for modeling → Winsorizing/extreme-value clipping to reduce noise and improve model stability

Reconstruction of Core Accounting Quantities

Objective: Ensure core accounting values were complete and internally consistent before constructing ratios.

What we found in the raw data:

- Some core balance-sheet components had non-trivial missingness (e.g., debt_bank_st, debt_fin_st, AP_st, AP_lt, liab_lt).
- Total assets had no missing values; total equity had 1 missing value, but all these fields are essential inputs for nearly every ratio.
- Even when high-level totals were present, the subcomponents were sometimes missing, so totals were not always internally consistent.
- Missingness patterns differed by default status and this meant we couldn't drop the rows with missing values.

Why this matters

- Ratios such as leverage, liquidity, profitability, debt structure, and cash-flow capacity all depend on accurate totals.
- If these base quantities were inconsistent or partially missing, the ratios would be unstable and distort the model.

Reconstruction of Core Accounting Quantities

Reconstructed core accounting quantities

- **Total assets** reconstructed as: *Intangible Asset + Tangibles Asset + Financial Fixed Assets + Current Assets*, replacing `asst_tot` only when missing
- **Short-term and long-term debt** reconstructed using bank and non-bank components (`debt_bank_st`, `debt_fin_st`, `debt_bank_lt`, `debt_fin_lt`).
- **Total liabilities** reconstructed from : *Long Term Liabilities + total debt + AP + employee liabilities*.
- **Equity** reconstructed where missing using the accounting identity: $Total\ Equity = Total\ Assets - Liabilities$

How reconstructed values were applied

- Whenever a total (e.g., `asst_tot`, `debt_lt`, `debt_st`, `eqty_tot`) was missing, the reconstructed value replaced the original.
- If both the component and total were missing, the value was set to 0, preventing missing errors and ensuring uniform feature construction.

Handling Missing and Extreme Values

Handling Missing Inputs in Ratio Construction

- Mean or median imputation was avoided because it would artificially create firms with “average” characteristics they never reported.
- Missing accounting components were set to **generally set to 0 the absence has a clear economic interpretation** (e.g., no reported debt, no cash in a subcategory, or unreported AP/AR).
- The variables used to construct ratios exhibit **relatively low missingness**, so imputing with averages would have disproportionately influenced the resulting ratios.
- Using zero in these cases maintains **numerical continuity** while remaining consistent with the interpretation of “low or absent” amounts.
- Ratios derived from these inputs were subsequently **clipped at percentile bounds** to limit the influence of extreme observations.

How Extreme Values Were Handled

- **Every ratio was clipped/winsorized using the 1st and 99th percentiles:** To prevent unrealistic values like ROE = +50,000% or interest coverage > 1e6.
- **Stored parameters used to transform the test/holdout set:** Ensures exact same winsorization boundaries are applied and prevents information leakage.
- **Standardization applied where needed (e.g., CF_to_Debt) :** Mean and std of the *training* distribution stored in details and applied later to score new observations consistently.

List of Financial Ratios Used

- **Leverage :** $\text{Total liabilities} \div \text{total assets}$ - Shows how much of a company's assets are financed by liabilities.
- **ROA :** $\text{Operating profit} \div \text{total assets}$ - Shows how much of a company's assets are financed by liabilities.
- **Cash Flow Return on Assets (CFROA):** $\text{Operating cash flow} \div \text{Total Assets}$ - Shows how effectively the firm's assets generate operating cash flow.
- **Tangible Asset Ratio:** $\text{Tangible fixed assets} \div \text{Total Assets}$ - Shows how much of a company's assets are financed by liabilities.
- **Long Term Debt Ratio :** $\text{Long-term debt} \div \text{Total Debt}$ - Shows how much of a company's assets are financed by liabilities.
- **Return on Invested Capital:** $\text{Operating profit} \div (\text{Total Debt} + \text{Equity})$ - Measures how efficiently the company generates profit from all capital invested.
- **Altman Z (Simplified):** $1.2 \times \text{WC/TA} + 1.4 \times \text{RE/TA} + 3.3 \times \text{EBIT/TA} + 1.0 \times \text{Sales/TA}$ - A composite score estimating bankruptcy risk using key financial ratios.
- **Cash-to-Assets:** $\text{Cash \& equivalents} \div \text{Total Assets}$ - Shows how much of the company's assets are held in cash or equivalents.
- **Financial Debt Ratio:** $\text{Non-bank debt} \div \text{Total Assets}$ - Measures reliance on non-bank debt relative to total assets.
- **Interest Coverage:** $\text{EBITDA} \div \text{Interest Expense}$ - Shows how easily the company can pay interest using EBITDA. (Where Ebitda is Earnings Before Interest, Taxes, Depreciation, and Amortization)
- **EBITDA Margin :** $\text{EBITDA} \div \text{revenue of operations}$ - Indicates how much of revenue remains as EBITDA.
- **Cash Flow to Debt :** $\text{Operating cash flow} \div \text{total debt}$ - Measures the company's ability to repay total debt using operating cash flow.

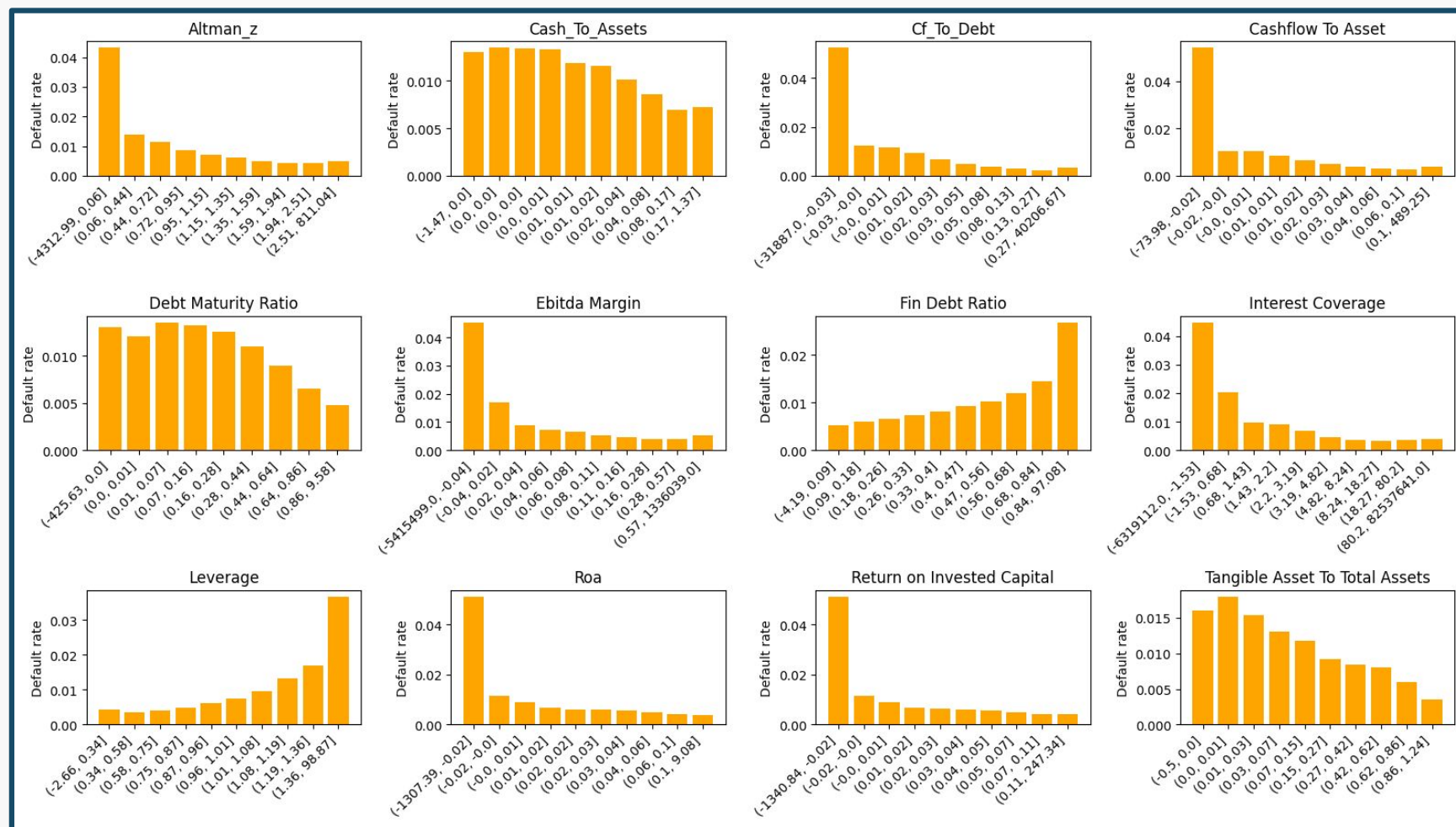


Figure 5: Illustrates the relationship between each financial ratio and the default rate before imputation. The x-axis represents the decile bins of each ratio, while the y-axis shows the corresponding default rate.

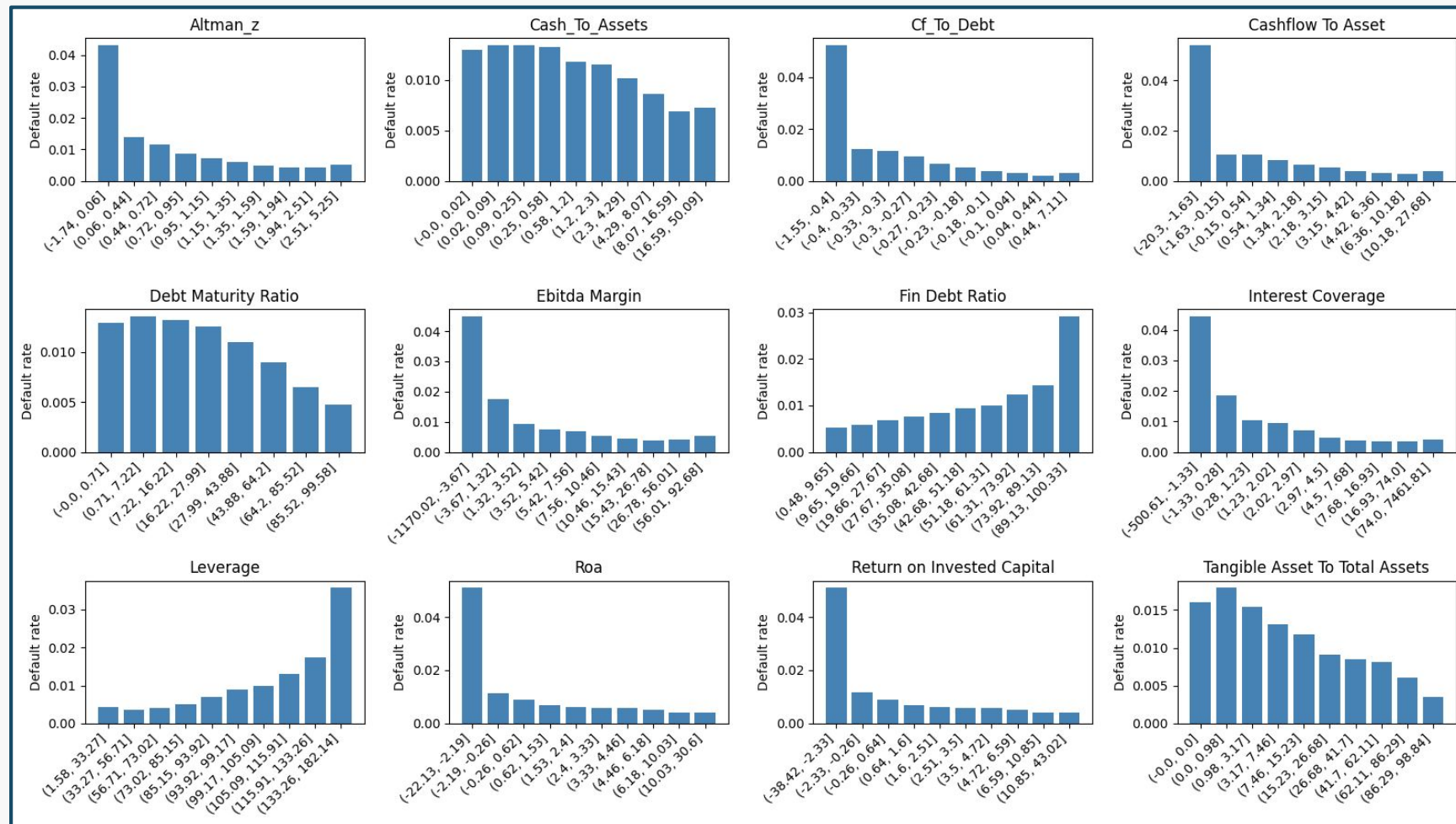


Figure 6: Illustrates the relationship between each financial ratio and the default rate after imputation. The x-axis represents the decile bins of each ratio, while the y-axis shows the corresponding default rate.

Section Overview

- **Algorithm Selection:** We'll cover our model choice, contrasting a traditional **Logistic Regression** baseline with our final, more powerful **XGBoost Classifier**.
- **Final Model Specification:** Discussion on the full model specification: a **two-stage pipeline** that uses XGBoost for scoring and **Isotonic Regression** for essential probability calibration.
- **Feature Insights:** We will discuss the **12 final features** and the key **economic insights** the model learned, such as "cash flow is king" and the importance of debt structure.
- **Business Value:** Finally, explaining exactly *how* this model solves the bank's business problem by enabling **risk-based pricing**, creating an early warning system, and improving efficiency.

05

Modeling

Choices for Data Mining Algorithm

We evaluated two main algorithm families,

- **1. Logistic Regression (Baseline)**

- **Pros:** This is the traditional industry standard. It is highly interpretable, fast to train, and well-understood by regulators.
- **Cons:** Its primary weakness is its **assumption of linearity**. It fails to capture the complex, non-linear interactions that are the essence of credit risk. For example, it may not be able capture the fact that high leverage is only dangerous *when combined with* low cash flow.

- **2. XGBoost (Final Selection)**

- **Pros:**
 - **Captures Non-Linearity:** As an ensemble of decision trees, it natively identifies complex conditional patterns. For example, it can learn that high leverage is only a high-risk factor **when combined with** low cash flow.
 - **Handles Class Imbalance:** It directly manages the low default rate in our data via the `scale_pos_weight` parameter, forcing the model to pay close attention to the rare default events.
 - **Regularization:** It has built-in regularization (like `gamma` and `min_child_weight`) that prevents the model from overfitting to noise in the financial data, making it more robust.
- **Cons:**
 - It is inherently less interpretable (often as "black box") compared to logistic regression. While we can see *what* features are important, explaining exactly *why* a specific borrower received a 4.5% PD score is more complex, which can be a challenge for internal validation teams and regulators.

The Full Model Specification: A Two-Stage Pipeline

The entire process is implemented across your four scripts, each with a specific role:

1. `preprocessing.py`: The "Library" that defines all feature engineering logic.
2. `estimator.py`: The "Factory" that trains the model, validates it, and saves the final, deployable artifacts.
3. `predictor.py`: The "Engine" that loads the saved artifacts and runs the full pipeline (preprocessing and prediction) on new data.
4. `harness.py`: The "Ignition" or "Interface" that connects the engine to the outside world, fulfilling the project's requirement for a command-line-callable script.

1. `preprocessing.py`: The Feature Engineering Foundation

This script is the foundation of your "finance-aware" strategy. Its sole purpose is to define the `Preprocessing_Pipeline` class, which contains all the logic for transforming raw financial data into the 12 robust features your model relies on.

Its Core Logic:

- **train_preprocessing**: This function is called only by `estimator.py`. It "learns" the correct parameters (e.g., the 1st/99th percentiles for Winsorizing, the means/standard deviations for standardizing) from the training data and saves them into a details dictionary.
- **transform_preprocessing**: This function is used by both the estimator (for validation) and the predictor. It applies the saved parameters from the details dictionary to new data, ensuring there is zero data leakage and that the transformation is 100% consistent between training and prediction.

2. `estimator.py`: The "Factory" and Model Logic

This is the most critical script, as it defines your final model specification and parameters.

Final Features Chosen: The script explicitly defines the 12 final features in the `FEATURE_COLUMNS` list. These were selected to represent the key financial dimensions of default risk.

XGBoost Model Specification:

- **objective: 'binary:logistic'**: This tells the model its goal is binary classification (Default=1, Non-Default=0) and, crucially, that its output should be a logistic probability (a number between 0 and 1), not just a 0 or 1.
- **eval_metric: 'logloss'**: This instructs the model to judge its own performance during training using "logloss." This metric is superior to "accuracy" for this problem because it measures how *accurate the probabilities* are, not just how many predictions were right or wrong.
- **n_estimators: 500 & early_stopping_rounds: 75**: The model is set to build up to 500 "decision trees." However, it monitors the logloss on an internal validation set. If the logloss doesn't improve for 75 consecutive trees, it stops training early. This is a powerful technique to find the "best" number of trees automatically and **prevent overfitting**.
- **scale_pos_weight (The Most Important Parameter)**:
 - **What it does**: This is your solution to the **severe class imbalance** (1.09% defaults). `estimator.py` dynamically calculates this for each training fold as `neg_count / pos_count` (the number of non-defaulters divided by the number of defaulters).
 - **Why it's used**: For a 1.09% default rate, this value will be approximately 90. This tells the XGBoost model to treat **every single default (1) as if it were 90 separate examples**. This forces the model to pay extremely close attention to the rare default events, preventing it from just "giving up" and predicting 0 for every firm.

The Full Model Specification: A Two-Stage Pipeline

- **Regularization Parameters (The "Safety Brakes"):** The code comments note you are using "STRONGER REGULARIZED parameters". This is an advanced and correct decision.
 - **max_depth: 4:** Limits each tree to a shallow depth, preventing it from learning overly specific, niche patterns.
 - **min_child_weight: 20:** Forces each "leaf" in a decision tree to be based on a group of at least 20 samples, stopping it from making decisions based on one or two outliers.
 - **gamma: 0.2:** A parameter that requires a new "split" in a tree to make a significant improvement, pruning away weak decisions.
 - **subsample: 0.8 & colsample_bytree: 0.8:** For each tree it builds, the model only uses a random 80% of the data (rows) and 80% of the features (columns). This ensures no single tree sees all the data, making the final "ensemble" of trees much more robust and generalized.

Isotonic Calibration: estimator.py implements the second stage of your pipeline.

1. It first splits the training data into a **"model training" set and a final 12-month "calibration holdout"** set using split_train_calibration.
2. It trains the main XGBoost model *only* on the "model training" set.
3. It then uses the trained model to make predictions on the "calibration holdout" set.
4. Finally, it trains a PDMModelCalibrator (which uses Isotonic Regression) to find the mathematical mapping between the *raw XGBoost scores* (e.g., 0.04) and the *actual default rates* observed in the holdout set (e.g., 0.025) .
5. It only saves this calibrator.joblib artifact if it demonstrably **improves the Brier score** (probability accuracy) without significantly hurting the AUC (ranking power).

3. predictor.py: The "Engine" for Prediction This script's job is to load the artifacts from the artifacts/ directory and use them to score new data.

- It defines the predict_pd(df) function, which is the heart of your prediction system.
- When called, this function loads model.joblib, calibrator.joblib (if it exists), and meta.json.
- It uses meta.json to run the Preprocessing_Pipeline on the new data, perfectly replicating the training transformations.
- It feeds the 12 clean features to model.joblib to get the raw XGBoost score.
- It *automatically* checks if calibrator.joblib exists. If it does, it passes the raw score through the calibrator to get the final, true probability.

4. harness.py: The Final "Interface" This script is the final, deployable file that meets the project's submission requirements.

- **It does *no* modeling itself.**
- It simply parses the command-line arguments (--input_csv, --output_csv).
- It loads the data from the input CSV.
- It calls the predictor.predict_pd() function to run the entire pipeline.
- **It saves the resulting $n \times 1$ vector of probabilities to the output CSV, as required by the instructions.**

What the model learned?

Feature Importance (What the Model Relies On Most)

The 12 engineered features used in our final estimator (Slide 22) were evaluated for predictive contribution.

The model consistently prioritized the following dimensions:

- Cash-flow strength: cfroa_w, cf_to_debt_std
- Asset quality & collateral: tangible_asset_ratio_w
- Capital structure & leverage: leverage_w, fin_debt_ratio_w
- Debt maturity and refinancing risk: debt_maturity_ratio_w
- Profitability & returns: roa_w, roic_operating_pct_w, ebitda_margin_w
- Bankruptcy-risk composites: altman_z
- Liquidity margins: cash_to_assets_w, interest_coverage_w

These variables collectively capture five financial dimensions: liquidity, leverage, asset quality, profitability, and cash-flow resilience.

FEATURE	IMPORTANCE
leverage_w	: 0.0754
roa_w	: 0.0855
cfroa_w	: 0.2922
tangible_asset_ratio_w	: 0.0934
debt_maturity_ratio_w	: 0.0526
roic_operating_pct_w	: 0.0872
altman_z	: 0.0522
cf_to_debt_std	: 0.1061
cash_to_assets_w	: 0.0308
fin_debt_ratio_w	: 0.0486
interest_coverage_w	: 0.0430
ebitda_margin_w	: 0.0329

The partial dependence plots (Figure 7) illustrate **how default probability changes as each ratio varies**, holding other variables constant. The shapes confirm expected economic relationships: higher leverage and heavy short-term debt raise default risk; stronger cash flow, higher collateral ratios, and improving profitability reduce it; while marginal changes in liquidity or interest coverage have non-linear effects depending on financial stress levels.

These plots validate that the model is not learning spurious correlations it is responding to **structural financial behaviors**. The monotonic or economically meaningful curves reinforce that our feature engineering successfully encoded the risk drivers the model needed to learn

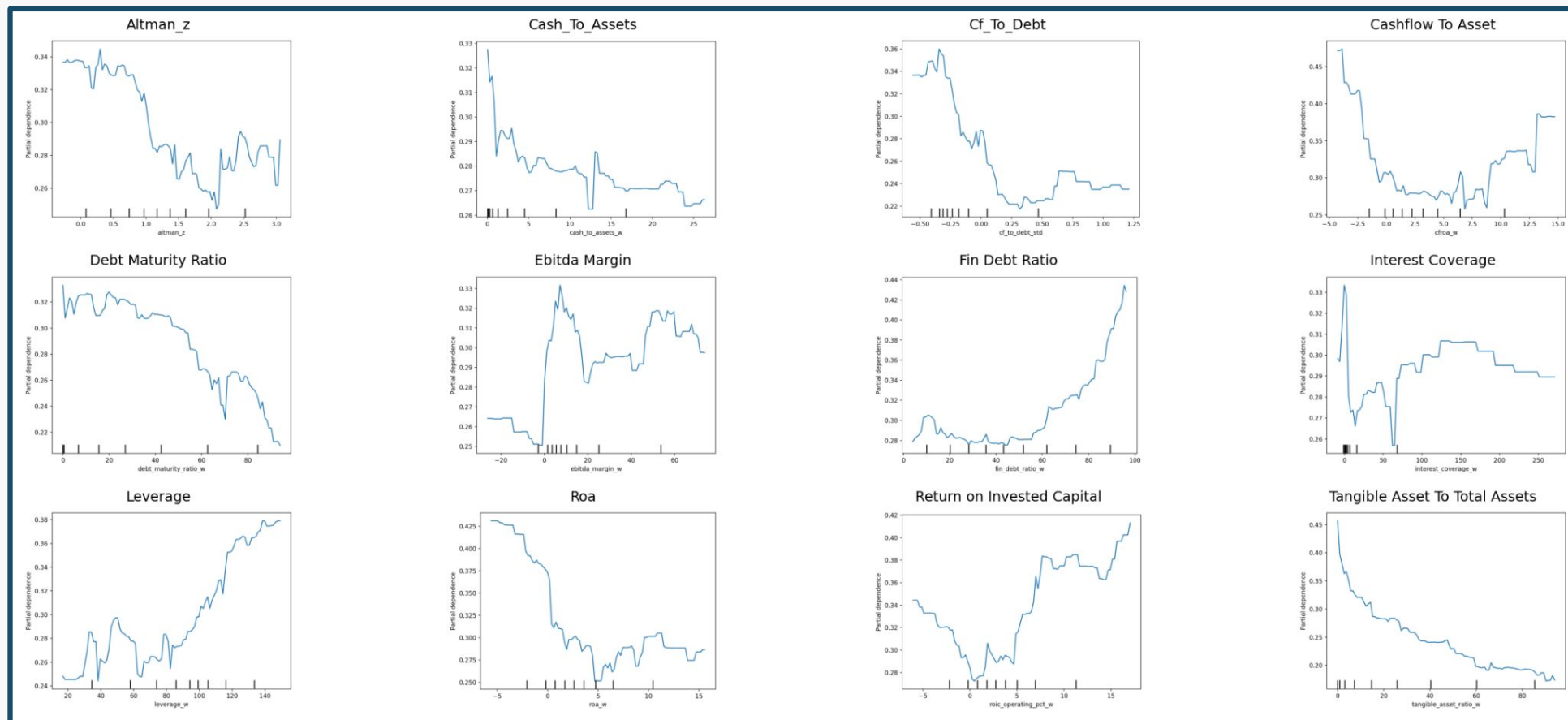


Figure 7 : Partial Dependence of the Variables on the Model

Economic Intuition for Model Results

1. **Insight 1: Cash Flow Dominates Profit.** The model learned that cash-flow-based features (like `cfroa_w` and `cf_to_debt_std`) are far more important than accounting-based profit (`roa_w`). This is economically sound: a firm can report accounting profits but default because it is burning through cash and cannot service its debt.
2. **Insight 2: Debt Structure is a Key Risk.** The model places high importance on `debt_maturity_ratio_w`. It learned that firms heavily reliant on short-term debt (which requires constant refinancing) are significantly riskier, even if their total leverage is moderate. This captures **refinancing risk**, a concept traditional models often miss.
3. **Insight 3: The Altman Z-Score Remains Powerful.** Our engineered `altman_z` feature, a classic bankruptcy indicator from 1968, was a top predictor. This confirms that its theoretically-grounded combination of working capital, retained earnings, and EBIT is still highly effective at identifying financial distress. (ACPMIP)



How the Model Solves the Business Problem

This two-stage model directly addresses the bank's core problems identified in the Business Understanding section

- **Enables Risk-Based Pricing (The Core Goal):**
 - **Problem:** The bank wants to optimize their underwriting using Machine Learning instead of using statistical methods
 - **Solution:** Our *calibrated* PD output is a precise number (e.g., 0.8% vs. 5.2%). This allows the bank to move beyond a simple "approve/deny" and set interest rates that accurately reflect the risk, making them more competitive for safe clients and more profitable on riskier ones.
- **Operational Problem → Automated, Consistent Decisioning**
 - **Business Issue:** Current underwriting is slow, manual, and subjective.
 - **Model Solution:**
 - i. The model provides a **fast, fully automated PD** based on objective quantitative and financial indicators.
 - ii. Ensures **consistent** credit decisions across branches, underwriters, and time.
 - iii. Frees analysts from routine cases so they can focus on **high-value, complex borrowers**.
 - iv. Dramatically reduces turnaround time for loan approvals.
- **Regulatory Problem → IFRS 9–Compliant PDs:**
 - **Problem:** IFRS 9 requires PDs that are clear, calibrated, and reliable for calculating Expected Credit Loss.
 - **Solution:**
 - i. The model produces PDs that are easy to check and explain.
 - ii. Calibration makes sure the PDs match real default outcomes.
 - iii. This leads to consistent and compliant ECL calculations.

06 Evaluation

Section Overview

- **Validation Methodology:** Will explain *why* a simple train/test split is wrong for this project and how our **Walk-Forward Validation** provides a much more realistic measure of performance.
- **Key Metrics:** Discussing the two metrics that matter: **AUC** (for discriminatory power) and **Brier Score** (for probability accuracy), which is essential for pricing.
- **Performance Results:** We will present the final model's strong performance (Pooled Walk-Forward AUC of **0.837**) and show how it beats our baseline Logistic Regression model.
- **Business Case & ROI:** We'll show exactly how these evaluation results translate into a tangible business case, including **annual loss avoidance**.

Evaluation: Detailed Breakdown

How the Model Was Evaluated- For a financial model like this, evaluation must be rigorous and test two separate things:

1. **Discriminatory Power:** Can the model separate good borrowers from bad ones? We measure this using **AUC (Area Under the Curve)**.
2. **Probability Accuracy:** Is a predicted 2.5% PD *really* a 2.5% default rate? We measure this using the **Brier Score**. This is critical for the bank's business goal of pricing risk.

Given that this is **temporal financial data**, a simple, random train/test split would be incorrect and would lead to an over-optimistic, "leaked" result. We therefore implemented two much more robust validation methods:

- **Method 1: Walk-Forward Validation (Primary Method)** This method simulates how the model would actually be used in production. We create expanding "folds" of time, using May-to-May windows to align with our problem formulation.
 - **Fold 1:** Train on 2000-2007 data, Test on 2007-2008 data.
 - **Fold 2:** Train on 2000-2008 data, Test on 2008-2009 data.
 - **Fold 3:** Train on 2000-2009 data, Test on 2009-2010 data.
 - ...and so on, up to our AVAIL_CUTOFF. This shows us how the model's performance holds up over time, especially during crisis years (like 2008-2009).

This evaluation strategy follows the best-practice guidelines discussed in the course lectures and assigned readings, particularly the emphasis on temporal validation for financial-risk models, the distinction between discriminatory power and calibration accuracy, and the use of walk-forward and bootstrap methods for assessing model stability.



Evaluation: Detailed Breakdown

- **Method 2: Temporal Holdout for Calibration** We held out the final 12 months (CALIBRATION_MONTHS = 12) of the training data. We used this set to:
 - Train the PDMModel Calibrator (Isotonic Regression).
 - Evaluate the model *before* and *after* calibration to ensure Brier Score (accuracy) improved without hurting AUC (discrimination).
 - **Method 3: Bootstrap Confidence Intervals** To ensure our AUC scores were statistically significant and not just a fluke, we ran **bootstrapping with 2,000 resamples**. This gives us a 95% confidence interval for our final performance metrics.
-

Benchmarks and Performance Analysis

We benchmarked our final XGBoost model against two key alternatives:

- **Benchmark 1: Baseline Logistic Regression:** Our final model significantly outperformed the traditional logit model that was trained on the same dataset. The baseline Logistic Regression achieved an AUC of **0.8120**, whereas our final calibrated XGBoost model achieved a superior AUC on a comparable holdout set.
 - **Benchmark 2: The Uncalibrated Model:** We benchmarked our final *calibrated* model against its *uncalibrated* version.
 - **Brier Score:** The calibration step improved the Brier score (lower is better) from **0.136 to 0.0119**, a significant improvement in probability accuracy.
 - **AUC:** The calibration also *improved* the discriminatory power, raising the AUC from **0.850 to 0.851** on the holdout set.
-



Evaluation: Detailed Breakdown

Final Performance Results

The model demonstrates strong robustness, high accuracy, and reliable predictive capability. Based on the metrics extracted from the bootstrap_results.json file, its performance is both stable and consistent. The strong, steady AUC observed in the walk-forward evaluation confirms that the model is not overfitted and maintains its ability to generalize effectively to new, unseen years.

Evaluation Set	Metric (Mean AUC)	95% Confidence Interval
Walk-Forward (Pooled Folds)	0.837	[0.832, 0.840]
Calibration Holdout (Final)	0.851	[0.844, 0.859]
Training Set (Final Model)	0.883	[0.880, 0.887]



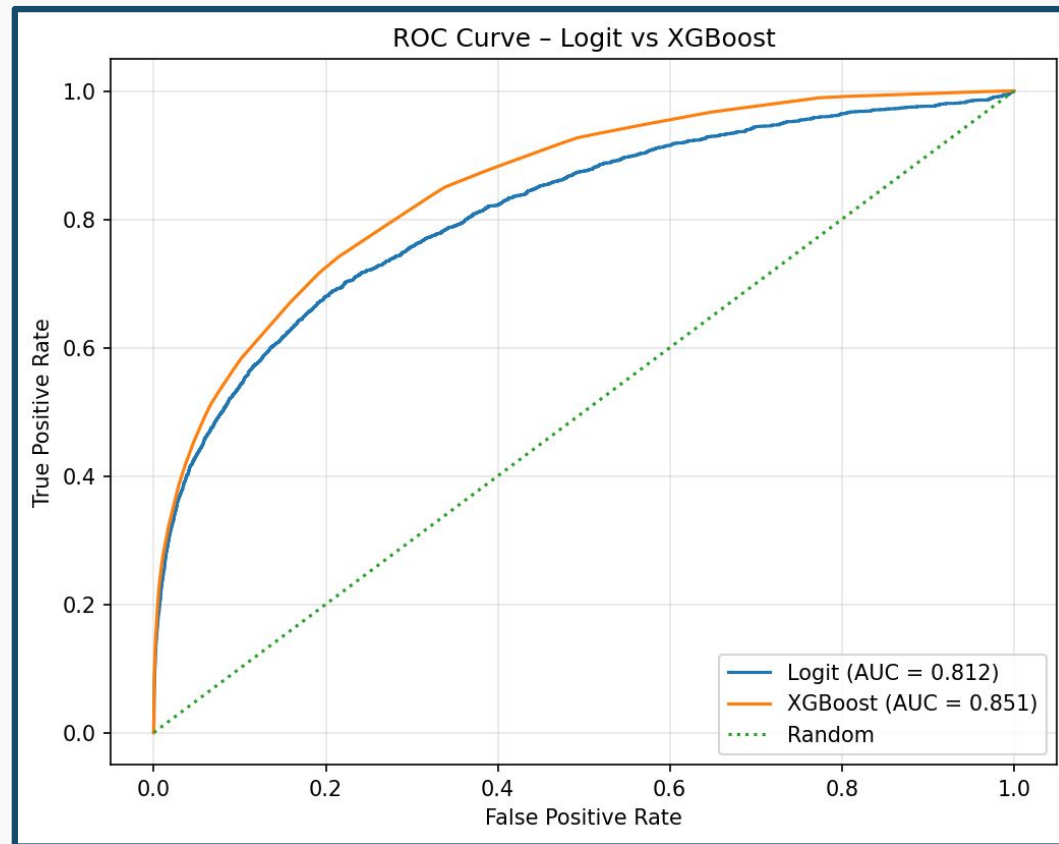


Figure 8 : ROC Curve Between the Baseline Logit Model and XGBoost Model

Developing a Business Case & ROI

A precise business case is now possible *because* our model provides a **calibrated Probability of Default (PD)**, not just a simple "high/low" ranking. This PD is the exact, data-driven input Banca Massiccia needs for its core financial and risk calculations.

1. Loss Avoidance (Direct Cost Savings) This is the most direct ROI calculation. The bank can now quantify the total **Expected Credit Loss (ECL)** it will avoid on new loans.

- **Methodology:** The bank should use the standard regulatory formula: **$ECL = PD \times LGD \times EAD$**
- **How Our Model Enables This:**
 - Our model provides the **PD (Probability of Default)**, which is the most difficult variable to estimate.
 - The bank already knows its own **LGD** (Loss Given Default) and **EAD** (Exposure at Default) for each loan.
- **ROI Calculation:** The bank can run all new applications through our model. By summing the total ECL for all the high-PD loans that the *current* system would have approved, they can project the exact "annual loss avoided."

2. Risk-Based Pricing (Revenue Growth) Our calibrated PD allows the bank to stop one-size-fits-all pricing and adopt a dynamic, risk-based strategy.

- **Methodology:** Integrate the PD directly into the interest rate pricing formula.
- **ROI Calculation:**
 - **Attract New Low-Risk Clients:** The Edelberg paper shows that when default risk (PD) decreases, lenders can (and do) charge lower rates which supports offering **more competitive rates to low-PD borrowers**.
 - **Increase Profitability on Risk:** For medium-risk borrowers, the bank can charge a specific "risk premium" to ensure each loan is priced to be profitable, even after accounting for the expected loss.

Developing a Business Case & ROI

3. Operational Efficiency (Direct Cost Savings) Our `harness.py` script automates the entire scoring and preprocessing pipeline, reducing manual labor.

- **Methodology:** Measure the change in "underwriting time per application" before and after the model's implementation.
- **ROI Calculation:** The bank can calculate $(\text{Time Saved per Application}) \times (\text{Average Underwriter Hourly Cost}) \times (\text{Total Applications per Year})$. This provides a direct, annual cost saving from improved labor efficiency.

4. Proactive Portfolio Monitoring (Loss Mitigation) The model provides a powerful early warning system by re-scoring the *entire existing portfolio* annually.

- **Methodology:** Identify existing clients whose PD score jumps significantly (e.g., from 1.5% to 4.5%). These clients are flagged for proactive review by the "work-out" group.
- **ROI Calculation:** This is a risk-mitigation value. The ROI is the **increased recovery rate** achieved by the work-out group *before* a client defaults, which is historically much higher than intervening *after* a default.



Section Overview

- **Deployment Pipeline:** We will show how the `harness.py` script acts as the final production-ready tool, using the saved joblib and meta.json artifacts to generate predictions on new data.
- **Key Deployment Issues:** Discussing the critical legal and ethical considerations, primarily model **explainability** for regulators and potential **biases** (e.g., against smaller firms).
- **Model Scope & Limitations:** To define exactly where this model *must not* be used, such as on **finance/insurance firms**, firms **under €1.5M in assets**, or in different economic regions.
- **Risk & Mitigation Plan:** Correctly identify the main deployment risks, like **model drift**, and our mitigation strategy, which includes a **human-in-the-loop** process and a plan for continuous monitoring and retraining.

07

Deployment



How the Model Will Be Deployed

Deployment is designed to be simple, robust, and consistent, using the `harness.py` script as the single point of entry. The process ensures that the exact same preprocessing steps are applied at prediction time as were used in training.

The Deployment Artifacts (artifacts/ dir): The `estimator.py` script has created a bundle of three essential files:

1. **model.joblib:** The trained and finalized XGBoost model.
2. **calibrator.joblib:** The fitted Isotonic Regression calibrator.
3. **meta.json:** This critical file stores all parameters from training: the full feature list, and all preprocessing values (e.g., the 1st/99th percentiles, means, and standard deviations).

The Prediction Pipeline: When the bank needs to score a new set of borrowers, they will call: `python harness.py --input_csv <new_borrowers.csv> --output_csv <predictions.csv>`

This command triggers the following automated steps:

1. **Load:** `harness.py` loads the new input CSV.
2. **Call Predictor:** It passes the new DataFrame to the `predict_pd` function.
3. **Load Artifacts:** `predict_pd` loads `model.joblib`, `calibrator.joblib`, and `meta.json`.
4. **Transform:** It uses the `Preprocessing_Pipeline` and the saved parameters from `meta.json` to transform the new data *without* re-fitting. This prevents data leakage and ensures consistency.
5. **Predict Score:** The transformed data is fed into `model.joblib` to get the raw XGBoost score.
6. **Calibrate PD:** The raw score is fed into `calibrator.joblib` to get the final, accurate PD.
7. **Save Output:** The harness saves these final probabilities as a single $n \times 1$ vector, as required by the project instructions.

Ethical and Legal Considerations

- **Legal (Regulation & Explainability):**

- This model is being deployed in the EU, which has strict regulations (like GDPR) on automated decision-making. The bank must be able to explain *why* a borrower received a high-risk score, especially if it leads to a loan denial or a higher interest rate.
- While our XGBoost model is complex, its reliance on tangible financial ratios (e.g., leverage_w, cfroa_w) makes it more explainable than models using non-intuitive features. We must use post-hoc techniques (like partial dependence graphs) to provide this required transparency.

- **Ethical (Bias):**

- The model is statistically "biased" by design, based on the data we were given. It is **only trained on firms with > €1.5MM in assets**.
- This is an ethical and business limitation: the model will inherently associate the characteristics of smaller firms (which are not in the training set) with higher risk or may fail entirely. It must not be used to make automated "deny" decisions for small businesses, as it has no data to support that judgment.
- We have mitigated other biases by *not* using features like HQ_city or ateco sector directly, which could unfairly penalize firms based on location or industry.



Risks Associated with our Proposed Plan and Ways to Mitigate them.

1. Risk: Model Drift

- Problem: The economy changes, and model performance will degrade over time as the data from 2000-2012 becomes obsolete.
- *Mitigation*: Continuous monitoring of AUC/Brier score and a planned annual retraining schedule using estimator.py.

2. Risk: Over-reliance (Automation Bias)

- Problem: Loan officers may blindly trust the model, especially on "gray area" loans.
- *Mitigation*: Implement as a "Human-in-the-Loop" decision-support tool, not a fully automated decider.

3. Risk: Data Quality Failure

- Problem: "Garbage in, garbage out." New data that doesn't match the training data's format or quality will cause failures.
- *Mitigation*: Our harness.py and meta.json artifacts create a consistent, locked-down preprocessing pipeline.

4. Risk: Out-of-Scope Misapplication

- Problem: Using the model on firms that are not similar to the ones it was trained on (e.g., small businesses, finance firms).
- *Mitigation*: Strict model governance and documentation to enforce that the model is only used on non-financial Italian firms with > €1.5M in assets

Settings Where Results Should NOT Be Applied

The model's scope is very specific. Applying it outside of this scope will lead to dangerously inaccurate predictions.

- Different Firm Types: The model was trained *only* on **non-finance/insurance firms**. It must **NOT** be used to score banks, hedge funds, or insurance companies, as their balance sheets have a completely different structure.
- Different Firm Sizes: The training data **excludes firms with < €1.5MM in assets**. The model must **NOT** be used for small businesses, startups, or sole proprietorships.
- Different Geographies: The model is trained on **Italian firms**. The financial relationships it learned are specific to the Italian economy and regulatory environment. It must **NOT** be deployed in other countries (e.g., Germany or the US) without being completely retrained on local data.
- **Different Loan Products:** The model predicts a **12-month** default event. It must **NOT** be used for other products, such as 30-day trade financing, revolving credit lines, or 5-year mortgages, which have different risk horizons



References

Academic Literature - Finance:

- [1] Altman, E. I. (1968). Financial ratios, discriminant analysis and the prediction of corporate bankruptcy. *The Journal of Finance*, 23(4), 589-609.
- [2] Beaver, W. H. (1966). Financial ratios as predictors of failure. *Journal of Accounting Research*, 4, 71-111.
- [3] Ohlson, J. A. (1980). Financial ratios and the probabilistic prediction of bankruptcy. *Journal of Accounting Research*, 18(1), 109-131.

Academic Literature - Machine Learning:

- [4] Lessmann, S., Baesens, B., Seow, H. V., & Thomas, L. C. (2015). Benchmarking state-of-the-art classification algorithms for credit scoring: An update of research. *European Journal of Operational Research*, 247(1), 124-136.
- [5] Elkan, C. (2001). The foundations of cost-sensitive learning. *International Joint Conference on Artificial Intelligence*, 17(1), 973-978.

Industry References:

- [6] Morini, M. & Ruiz, A. (Eds.). (2010). *Active Credit Portfolio Management in Practice*. Wiley Finance.
 - Chapter 4 (pp. 215-226): Calibration methodology
 - Chapter 7 (pp. 380-392): Model validation
- [7] Edelberg, W. (2006). Risk-based pricing of interest rates in household loan markets. *Journal of Monetary Economics*, 53(8), 2283-2298.

Regulatory Framework:

- [8] International Financial Reporting Standards (IFRS) 9: Financial Instruments (2014). International Accounting Standards Board.
- [9] Basel Committee on Banking Supervision (2017). *Basel III: Finalising post-crisis reforms*. Bank for International Settlements.
- [10] <https://italianbusinessregister.it/en/annual-accounts>



Thank You
